

Міністерство освіти і науки України
Національний технічний університет України «Київський політехнічний
інститут імені Ігоря Сікорського»
Факультет інформатики та обчислювальної техніки
Кафедра інформатики та програмної інженерії

Звіт
з практичної роботи № 8 з дисципліни
«Основи програмування. Частина 2.
Модульне програмування»
«ВІДНОШЕННЯ МІЖ КЛАСАМИ ТА ОБ'ЄКТАМИ»
Варіант 13

Виконав студент ІП-55 Дзьобань Володимир Валентинович
(шифр, прізвище, ім'я, по батькові)

Перевірів Куценко Микита Олександрович
(прізвище, ім'я, по батькові)

Київ 2025

ВІДНОШЕННЯ МІЖ КЛАСАМИ ТА ОБ'ЄКТАМИ

Мета – дослідити типи відношень між класами та об'єктами в ООП, навчитися проектувати об'єктно-орієнтовану модель предметної галузі.

Завдання

1. Вивчити типи відношень між класами в ООП.
2. Спроекувати об'єктно-орієнтовану модель предметної галузі згідно з варіантом, визначивши необхідні для цього класи та їх структуру.
3. Вимоги до проектування:
 - розробити не менше 6 типів даних;
 - застосувати всі базові принципи ООП;
 - застосувати всі види відношень;
 - застосувати обробку виключень, де це є необхідним;
 - дотримуватись єдиної конвенції найменувань та принципів написання "чистого" коду;
 - код дозволяється коментувати лише xml-коментарями.
4. Написати програму, в якій реалізувати попередньо спроектовану об'єктно-орієнтовану модель.
5. Програмний інтерфейс, наприклад, введення\виведення з консолі, реалізовувати окремим проектом. Код програмного інтерфейсу має бути простим (демонструється використання класів предметної галузі шляхом створення об'єктів та їх застосування, відсутня перевірка коректності вводу, введення з консолі мінімальне або відсутнє взагалі).

Варіант 13

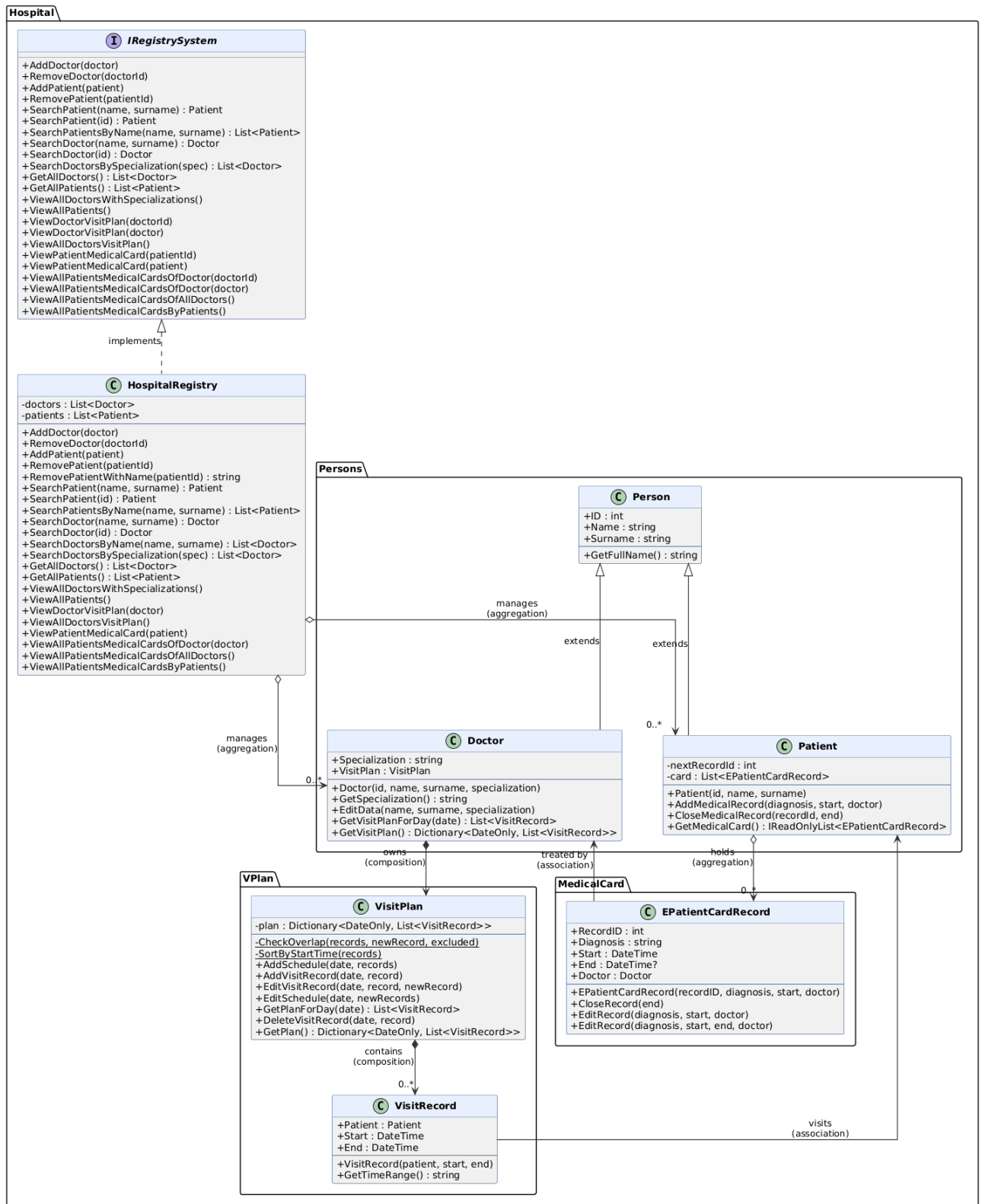
Реєстратура лікарні: облік хворих та запис до лікарів на прийом

Функціональні вимоги до програмного забезпечення

1. Управління лікарями
 - 1.1. Можливість додавати лікаря
 - 1.2. Можливість видаляти лікаря
 - 1.3. Можливість змінити дані про лікаря
 - 1.4. Можливість перегляду списку всіх лікарів та їх спеціалізацій
2. Управління хворими
 - 2.1. Можливість додавати хворого
 - 2.2. Можливість видаляти хворого
 - 2.3. Можливість ведення електронної картки хворого
3. Управління розкладом прийому
 - 3.1. Можливість додавати розклад
 - 3.2. Можливість змінювати розклад
 - 3.3. Можливість записувати хворого до лікаря на прийом
4. Пошук
 - 4.1. Можливість пошуку хворого за його даними (прізвище, ім'я)
 - 4.2. Можливість пошуку лікаря
 - 4.3. Можливість отримання розкладу лікаря на певний час

Розв'язок

UML діаграма класів:



У ході виконання лабораторної було створено 8 класів та застосовано усі види відношень між ними.

У проєкті Hospital знаходиться уся реалізація проєкту відповідно варіанту та створений UML діаграмі класів.

У проєкті ProgramApp реалізовано використання Hospital та відповідний інтерфейс для роботи з ним.

Посилання на GitHub репозиторій:

https://github.com/VolodymyrDzoban/OP_Lab8.git

Висновки

У ході виконання лабораторної роботи було досліджено типи відношень між класами та об'єктами в об'єктно-орієнтованому програмуванні, а також спроектовано та реалізовано об'єктно-орієнтовану модель предметної галузі «Реєстратура лікарні: облік хворих та запис до лікарів на прийом» (Варіант 13).

В рамках роботи розроблено 8 типів даних, що перевищує мінімальну вимогу в 6 типів: Person, Doctor, Patient (пакет Hospital.Persons), VisitPlan, VisitRecord (пакет Hospital.VPlan), EPatientCardRecord (пакет Hospital.MedicalCard), HospitalRegistry та інтерфейс IRegistrySystem (пакет Hospital).

В розробленій моделі застосовано всі види відношень між класами:

- Узагальнення (наслідування) — класи Doctor та Patient успадковують спільні атрибути (ID, Name, Surname) та метод GetFullName() від базового класу Person, розширюючи його специфічною функціональністю.
- Реалізація — клас HospitalRegistry реалізує інтерфейс IRegistrySystem, що визначає контракт системи реєстрації та забезпечує можливість заміни конкретної реалізації без зміни коду, що її використовує.
- Композиція — клас Doctor містить об'єкт VisitPlan, який створюється разом із лікарем і не може існувати без нього; VisitPlan у свою чергу містить об'єкти VisitRecord, що також не існують поза розкладом.
- Агрегація — HospitalRegistry агрегує колекції Doctor та Patient: реєстр керує їх списками, але самі об'єкти лікарів і пацієнтів можуть існувати незалежно. Клас Patient агрегує записи EPatientCardRecord у вигляді електронної медичної картки.

- Асоціація — VisitRecord зберігає посилання на Patient (запис до лікаря пов'язаний з конкретним пацієнтом); EPatientCardRecord зберігає посилання на Doctor (медичний запис пов'язаний з лікарем, що його відкрив). Обидва є однонаправленими асоціаціями.

Також у роботі застосовано всі базові принципи ООП: інкапсуляція реалізована через модифікатори доступу та властивості з private set; наслідування — через ієрархію Person → Doctor/Patient; поліморфізм — через перевантаження методів (SearchDoctor, SearchPatient, EditRecord, ViewPatientMedicalCard тощо) та реалізацію інтерфейсу; абстракція — через інтерфейс IRegistrySystem, що приховує деталі реалізації реєстру.

Обробку виключень реалізовано в усіх критичних місцях: ArgumentNullException при передачі null-аргументів, ArgumentException при некоректних даних (дублювання ID, некоректний часовий інтервал, накладання записів у розкладі, запис і кінець прийому в різних днях), InvalidOperationException при спробі виконати неможливу операцію (запис не знайдено, лікар/пацієнт відсутній, повторне закриття медичного запису).

Дотримано конвенцію найменувань C# (PascalCase для публічних членів, camelCase для параметрів), принципи чистого коду (єдина відповідальність методів, відсутність дублювання логіки) та вимогу документування виключно XML-коментарями.

Практично реалізовано повний функціонал варіанту 13: управління лікарями та пацієнтами, ведення електронних медичних карток з автоматичною нумерацією записів усередині картки кожного пацієнта, управління розкладом прийому з перевіркою накладань та автоматичним сортуванням за часом, а також пошук за різними критеріями.